

Glossary:

Diabetes: “Metabolic disorder in which the body's ability to produce or respond to the hormone insulin is impaired¹”.

Multilayer Perceptron (MLP): Feed forward neural network consisting of an input layer, at least one hidden layer and an output layer.

Support Vector Machine (SVM): Algorithm used to classify data by identifying the best line/hyperplane that maximises the gap between each class, in N dimensions².

Error/Loss Function: Mathematical procedure that calculates the error margin between the model's predicted versus actual value³.

Kernel Function: Technique that takes input data and transforms it into a suitable form of processing data⁴.

Grid Search: Method used to find the optimal hyper-parameters for the specified model.

Baseline Model: Initially trained model, used to understand how it behaves with default parameters.

Cross-Validation: Machine learning method used to judge how efficient is a model on unseen data⁵.

Hyper-parameters: Set of values, which vary based on the method used, that are going to be tuned to improve the model's performance.

Confusion Matrix: Matrix used to count the number of true/false (T/F) positives/negatives (P/N).

AUC-ROC: Value that shows how good a model is, in correctly ranking predictions.

Accuracy: $\frac{TP+TN}{TP+TN+FP+FN}$

Precision: $\frac{TP}{TP+FP}$

Recall: $\frac{TP}{TP+FN}$

F1-Score: $2 * \frac{Precision * Recall}{Precision + Recall}$

Training Time: How long it takes for the model to train.

Epoch: One complete pass of the entire training dataset.

Learning Rate: Rate at which the model learns in predicting the target variable.

Activation Function: Function that quantifies a node's output given its distinct inputs and their weight values⁷.

References:

¹diabetes, n. meanings, etymology and more | Oxford English Dictionary. (2024). *Oed.com*. [online] doi:

<https://doi.org/10.1093/OED/1061429003>.

²IBM (2023). *Support Vector Machine*. [online] IBM. Available at: <https://www.ibm.com/think/topics/support-vector-machine>.

³Richmond Alake (2023). *Loss Functions in Machine Learning Explained*. [online] Datacamp.com. Available at: <https://www.datacamp.com/tutorial/loss-function-in-machine-learning>.

⁴GeeksforGeeks. (2020). *Major Kernel Functions in Support Vector Machine (SVM)*. [online] Available at: <https://www.geeksforgeeks.org/major-kernel-functions-in-support-vector-machine-svm/>.

⁵Sharma, A. (2024). *Cross Validation in Machine Learning*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/cross-validation-machine-learning/>.

⁶Evidently AI Team (2024). *Accuracy vs. precision vs. recall in machine learning: what's the difference?* [online] www.evidentlyai.com. Available at: <https://www.evidentlyai.com/classification-metrics/accuracy-precision-recall>.

⁷Wikipedia Contributors (2018). *Activation function*. [online] Wikipedia. Available at: https://en.wikipedia.org/wiki/Activation_function.

Intermediate Results:

In the intermediate results, for the MLP model specifically, the code which implemented two hidden layers instead of only one, was run. Moreover, also the grid search to find the best combination of hyper-parameter values for that model, was constructed. The best parameters in this case were the following:

Learning rate=0.01; No. of hidden neurons in first hidden layer=40; No. of hidden neurons in second hidden layer=32; Momentum=0.9; Weight Decay=0.

This was not kept in the final code model because the best F1-Score in this model was less than the F1-Score of the model with a single hidden layer.

The code snippets of the model and the grid search can be found below.

```
class MLPModuleV2(nn.Module):
    def __init__(self, input_dim, hidden_dim1=64, hidden_dim2=32):
        super(MLPModuleV2, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim1)
        self.fc2 = nn.Linear(hidden_dim1, hidden_dim2)
        self.output = nn.Linear(hidden_dim2, 1)

    def forward(self, X):
        X = F.relu(self.fc1(X))
        X = F.relu(self.fc2(X))
        X = torch.sigmoid(self.output(X))
        return X.view(-1)

net_v2 = NeuralNetClassifier(
    MLPModuleV2,
    module__input_dim=X_train.shape[1],
    max_epochs=50,
    optimizer=torch.optim.SGD,
    criterion=nn.BCELoss,
    verbose=1,
    device='cpu')

param_grid_v2 = {
    'module__hidden_dim1': [40,32],
    'module__hidden_dim2': [32,24],
    'lr': [0.1,0.01,0.001],
    'optimizer__momentum': [0,0.9],
    'optimizer__weight_decay': [0,1e-4,1e-3]
}
```

```
from sklearn.model_selection import GridSearchCV

X_sub, _, y_sub, _ = train_test_split(X_train_scaled, y_train, test_size=0.9, stratify=y_train, random_state=42)

grid_v2 = GridSearchCV(net_v2, param_grid_v2, cv=3, scoring='f1', verbose=2, n_jobs=-1)
grid_v2.fit(X_sub, y_sub)

print("Best parameters:", grid_v2.best_params_)
print("Best F1 score :", grid_v2.best_score_)
```

Implementation Details:

One of the main implementation choices that was made was to decrease the number of possible hyper-parameter values in the grid search for the SVM model, specifically the 'gamma' and 'degree' parameters. This was done due to the fact that the grid search was taking too long to run, in particular the one with the 'poly' kernel when it was training with higher 'degree' values.

Something similar was done with the MLP model, where, in the grid search, the 'Adam' optimiser was not implemented. The reason for that was the fact that when this was tried to be employed, an error occurred.